杰理OTA升级(Flutter)收发接口介绍

版本	更新日期	更新内容	修改人
V1.0.0	2025/11/19	初始版本发布	李放

概述

杰理OTA升级APP是一款专为使用杰理芯片的设备设计的在线升级工具。它允许用户通过蓝牙对设备进行固件升级，以确保设备始终拥有最新的功能和安全修复。

1. Dart的发送层的接口(ble_method.dart)

1.1 初始化MethodChannel

```
static const MethodChannel _methodChannel = MethodChannel(
  'com.jieli.ble_plugin/methods',
);
```

1.2 开始扫描

```
  static Future<void> startScan() async {
  try {
    await _methodChannel.invokeMethod(BleMethodConstants.METHOD_START_SCAN);
  } on PlatformException catch (e) {
    print("Failed to start scan: ${e.message}");
    rethrow;
  }
}

使用示例: await BleMethod.startScan()
```

1.3 停止扫描

```
 static Future<void> stopScan() async {
  try {
    await _methodChannel.invokeMethod(BleMethodConstants.METHOD_STOP_SCAN);
  } on PlatformException catch (e) {
    print("Failed to stop scan: ${e.message}");
    rethrow;
  }
}

使用示例: await BleMethod.stopScan()
```

1.4 连接设备

```
static Future<void> connectDevice(int index) async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_CONNECT_DEVICE,
      {BleMethodConstants.ARG_INDEX: index},
    );
  } on PlatformException catch (e) {
    print("Failed to connect device at index $index: ${e.message}");
    rethrow;
  }
}

使用示例:
/// Connect to a device at the specified index
void _connectToDevice(int index) async {
  try {
    await BleMethod.connectDevice(index);
  } catch (e) {
    log("Failed to connect to device: $e");
    // Optionally show an error message to the user
  }
}
```

1.5 断开设备

```
  static Future<void> disconnectBtDevice(int index) async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_DISCONNECT_BT_DEVICE,
      {BleMethodConstants.ARG_INDEX: index},
    );
  } on PlatformException catch (e) {
    print("Failed to disconnect device at index $index: ${e.message}");
    rethrow;
  }
}

使用示例:
  /// Disconnect from a device at the specified index
void _disconnectBtDevice(int index) async {
  try {
    await BleMethod.disconnectBtDevice(index);
  } catch (e) {
    log("Failed to disconnect from device: $e");
    // Optionally show an error message to the user
  }
}
```

1.6 读取当前是否使用BLE通讯 (Android)

```
  static Future<bool> isBleWay() async {
  try {
    return await _methodChannel.invokeMethod(
          BleMethodConstants.METHOD_IS_BLE_WAY,
        ) ??
        true;
  } on PlatformException catch (e) {
    print("Failed to check if BLE way is used: ${e.message}");
    rethrow;
  }
}

 使用示例:await BleMethod.isBleWay()
```

1.7 设置是否使用BLE通讯 (Android)

```
static Future<void> setBleWay(bool isBle) async {
  try {
    await _methodChannel.invokeMethod(BleMethodConstants.METHOD_SET_BLE_WAY, {
      BleMethodConstants.ARG_IS_BLE: isBle,
    });
  } on PlatformException catch (e) {
    print("Failed to set BLE way: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.setBleWay(true)
```

1.8 读取是否使用SDK蓝牙连接 (IOS)

```
static Future<bool> isUseSdkBluetooth() async {
  try {
    return await _methodChannel.invokeMethod(
          BleMethodConstants.METHOD_IS_USING_SDK_BLUETOOTH,
        ) ??
        true;
  } on PlatformException catch (e) {
    print("Failed to check if sdk bluetooth is used: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.isUseSdkBluetooth()
```

1.9 设置是否使用SDK蓝牙连接 (IOS)

```
static Future<void> setConnectUsingSdkBluetooth(bool isUsingSDKBluetooth) async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_SET_USING_SDK_BLUETOOTH,
      {BleMethodConstants.ARG_IS_USING_SDK_BLUETOOTH: isUsingSDKBluetooth},
    );
  } on PlatformException catch (e) {
    print("Failed to use sdk bluetooth: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.setConnectUsingSdkBluetooth(true)
```

1.10 读取是否需要设备认证 (Android和IOS)

```
  static Future<bool> isUseDeviceAuth() async {
  try {
    return await _methodChannel.invokeMethod(
          BleMethodConstants.METHOD_IS_USE_DEVICE_AUTH,
        ) ??
        true;
  } on PlatformException catch (e) {
    print(
      "Failed to check if device authentication is used: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.isUseDeviceAuth()
```

1.11 设置是否需要设备认证 (Android和IOS)

```
static Future<void> setUseDeviceAuth(bool isAuth) async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_SET_USE_DEVICE_AUTH,
      {BleMethodConstants.ARG_IS_AUTH: isAuth},
    );
  } on PlatformException catch (e) {
    print("Failed to set device authentication: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.setUseDeviceAuth(true)
```

1.12 读取当前是否为HID设备 (Android)

```
static Future<bool> isHidDevice() async {
  try {
    return await _methodChannel.invokeMethod(
          BleMethodConstants.METHOD_IS_HID_DEVICE,
        ) ??
        false;
  } on PlatformException catch (e) {
    print("Failed to check if HID device: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.isHidDevice()
```

1.13 设置是否为HID设备 (Android)

```
static Future<void> setHidDevice(bool isHid) async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_SET_HID_DEVICE,
      {BleMethodConstants.ARG_IS_HID: isHid},
    );
  } on PlatformException catch (e) {
    print("Failed to set HID device: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.setHidDevice(true)
```

1.14 读取是否使用自定义回连方式 (Android)

```
static Future<bool> isUseCustomReConnectWay() async {
  try {
    return await _methodChannel.invokeMethod(
          BleMethodConstants.METHOD_IS_USE_CUSTOM_RECONNECT_WAY,
        ) ??
        false;
  } on PlatformException catch (e) {
    print(
      "Failed to check if custom reconnect way is used: ${e.message}"
    );
    rethrow;
  }
}

使用示例:await BleMethod.isUseCustomReConnectWay()
```

1.15 设置是否使用自定义回连方式 (Android)

```
static Future<void> setUseCustomReConnectWay(bool isCustom) async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_SET_USE_CUSTOM_RECONNECT_WAY,
      {BleMethodConstants.ARG_IS_CUSTOM: isCustom},
    );
  } on PlatformException catch (e) {
    print("Failed to set custom reconnect way: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.setUseCustomReConnectWay(true)
```

1.16 获取当前BLE MTU请求值 (Android)

```
static Future<int> getBleRequestMtu() async {
  try {
    return await _methodChannel.invokeMethod(
          BleMethodConstants.METHOD_GET_BLE_REQUEST_MTU,
        ) ??
        0;
  } on PlatformException catch (e) {
    print("Failed to get BLE MTU: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.getBleRequestMtu()
```

1.17 设置BLE MTU请求值(范围:23~509）(Android)

```
static Future<void> setBleRequestMtu(int mtu) async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_SET_BLE_REQUEST_MTU,
      {BleMethodConstants.ARG_MTU: mtu},
    );
  } on PlatformException catch (e) {
    print("Failed to set BLE MTU: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.setBleRequestMtu(500)
```

1.18 获取SDK版本号

```
static Future<String> getSdkVersion() async {
  try {
    return await _methodChannel.invokeMethod(
          BleMethodConstants.METHOD_GET_SDK_VERSION,
        ) ??
        'V?.?.?(?)';
  } on PlatformException catch (e) {
    print("Failed to get SDK version: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.getSdkVersion()
```

1.19 获取APP版本号

```
static Future<String> getAppVersion() async {
  try {
    return await _methodChannel.invokeMethod(
          BleMethodConstants.METHOD_GET_APP_VERSION,
        ) ??
        'V?.?.?(?)';
  } on PlatformException catch (e) {
    print("Failed to get APP version: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.getAppVersion()
```

1.20 读取日志文件目录路径

```
static Future<String> getLogFileDirPath() async {
  try {
    return await _methodChannel.invokeMethod(
          BleMethodConstants.METHOD_GET_LOG_FILE_DIR_PATH,
        ) ??
        '';
  } on PlatformException catch (e) {
    print("Failed to get log file directory path: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.getLogFileDirPath()
```

1.21 获取日志文件列表

```
static Future<void> getLogFiles() async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_GET_LOG_FILES,
    );
  } on PlatformException catch (e) {
    print("Failed to get log files: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.getLogFiles()
```

1.22 点击日志文件列表的索引

```
static Future<void> clickLogFileIndex(int logFileIndex) async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_LOG_FILE_INDEX,
      {BleMethodConstants.ARG_LOG_FILE_INDEX: logFileIndex},
    );
  } on PlatformException catch (e) {
    print(
      "Failed to click log file index $logFileIndex: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.clickLogFileIndex(1) // 1:当前logFileIdnex
```

1.23 删除全部的日志文件

```
static Future<bool> deleteAllLogFiles() async {
  try {
    return await _methodChannel.invokeMethod(
          BleMethodConstants.METHOD_DELETE_ALL_LOG_FILE,
        ) ??
        false;
  } on PlatformException catch (e) {
    print("Failed to delete all log files: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.deleteAllLogFiles()
```

1.24 分享日志文件

```
static Future<void> shareLogFile(int logFileIndex) async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_SHARE_LOG_FILE,
      {BleMethodConstants.ARG_LOG_FILE_INDEX: logFileIndex},
    );
  } on PlatformException catch (e) {
    print(
      "Failed to share log file at index $logFileIndex: ${e.message}");
    rethrow;
  }
}

使用示例:await BleMethod.shareLogFile(2) // 2:当前logFileIdnex
```

1.25 下载文件

```
static Future<void> downloadFile(String httpUrl) async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_DOWNLOAD_FILE,
      {BleMethodConstants.ARG_HTTP_URL: httpUrl},
    );
  } on PlatformException catch (e) {
    print("Failed to download file from $httpUrl: ${e.message}");
    rethrow;
  }
}

使用示例:
String httpUrl = '' // httpUrl是下载的链接
await BleMethod.downloadFile(httpUrl)
```

1.26 是否在OTA升级

```
static Future<bool> isOTA() async {
  try {
    return await _methodChannel.invokeMethod(
          BleMethodConstants.METHOD_TYPE_IS_OTA,
        ) ??
        true;
  } on PlatformException catch (e) {
    print("Failed to get Ota state: ${e.message}");
    rethrow;
  }
}

使用示例:
await BleMethod.isOTA()
```

1.27 读取OTA文件列表

```
static Future<void> readFileList() async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_READ_FILE_LIST,
    );
  } on PlatformException catch (e) {
    print("Failed to read OTA file list: ${e.message}");
    rethrow;
  }
}

使用示例:
await BleMethod.readFileList()
```

1.28 设置OTA文件列表选中的文件索引

```
static Future<void> setSelectedIndex(int pos) async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_SET_SELECTED_INDEX,
      {BleMethodConstants.ARG_POS: pos},
    );
  } on PlatformException catch (e) {
    print("Failed to set selected index $pos: ${e.message}");
    rethrow;
  }
}

使用示例:
await BleMethod.setSelectedIndex(1) // 1:当前选中升级的文件索引
```

1.29 删除OTA文件列表选中的文件索引

```
static Future<void> deleteOtaIndex(int pos) async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_DELETE_OTA_FILE_INDEX,
      {BleMethodConstants.ARG_POS: pos},
    );
  } on PlatformException catch (e) {
    print("Failed to delete OTA file index $pos: ${e.message}");
    rethrow;
  }
}

使用示例:
await BleMethod.deleteOtaIndex(1) // 1:当前选中删除的升级文件索引
```

1.30 检测外部存储权限环境

```
static Future<bool> tryToCheckStorageEnvironment() async {
  try {
    return await _methodChannel.invokeMethod(
          BleMethodConstants.METHOD_TRY_TO_CHECK_STORAGE_ENVIRONMENT,
        ) ??
        false;
  } on PlatformException catch (e) {
    print("Failed to check storage environment: ${e.message}");
    rethrow;
  }
}

使用示例:
await BleMethod.tryToCheckStorageEnvironment()
```

1.31 选择文件

```
static Future<void> pickFile() async {
  try {
    await _methodChannel.invokeMethod(BleMethodConstants.METHOD_PICK_FILE);
  } on PlatformException catch (e) {
    print("Failed to pick file: ${e.message}");
    rethrow;
  }
}

使用示例:
await BleMethod.pickFile()
```

1.32 开始OTA更新

```
static Future<void> startOTA(String path) async {
  try {
    await _methodChannel.invokeMethod(
        BleMethodConstants.METHOD_START_OTA,
        {BleMethodConstants.ARG_PATH: path}
    );
  } on PlatformException catch (e) {
    print("Failed to start OTA: ${e.message}");
    rethrow;
  }
}

使用示例:
String path = '' // path是ota升级的文件存储路径
await BleMethod.startOTA(path)
```

1.33 获取WiFi的IP地址(返回格式: "http://[ip]:[port]")

```
static Future<String> getWifiIpAddress() async {
  try {
    final String? ipAddress = await _methodChannel.invokeMethod<String>(
      BleMethodConstants.METHOD_GET_WIFI_IP_ADDRESS,
    );
    return ipAddress ?? 'Failed to get WiFi IP address';
  } on PlatformException catch (e) {
    print("Failed to get WiFi IP address: ${e.message}");
    rethrow;
  }
}

使用示例:
await BleMethod.getWifiIpAddress()
```

1.34 退出所有的Activity

```
static Future<void> popAllActivity() async {
  try {
    await _methodChannel.invokeMethod(
      BleMethodConstants.METHOD_POP_ALL_ACTIVITY,
    );
  } on PlatformException catch (e) {
    print("Failed to pop all activities: ${e.message}");
    rethrow;
  }
}

使用示例:
await BleMethod.popAllActivity()
```

2. Dart的接收层的接口(ble_event_stream.dart)

2.1 初始化baseStream和EventChannel

```
 static const EventChannel _eventChannel = EventChannel('com.jieli.ble_plugin/events');

// 核心广播流
// 单例模式:确保_baseStream只被初始化一次
static Stream<dynamic>? _baseStream;

// 提供一个公共的访问方法
static Stream<dynamic> get baseStream {
  _baseStream ??= _eventChannel.receiveBroadcastStream();
  return _baseStream!;
}
```

2.2 扫描状态流

```
  static Stream<String> get scanStateStream {
    return baseStream
        .where((event) => event is Map && event[BleEventConstants.KEY_TYPE] ==
BleEventConstants.TYPE_SCAN_DEVICE_LIST)
        .map((event) {
      final data = event[BleEventConstants.KEY_VALUE];
      return data[BleEventConstants.KEY_STATE] as String? ?? '';
    });
  }

  使用示例:
  StreamSubscription<String>? _scanStateSubscription;

  void _subscribeToScanStateStream() {
    _scanStateSubscription = BleEventStream.scanStateStream.listen(
      (state) {
        if (!mounted) return;

        setState(() {
          if (state == BleEventConstants.SCAN_STATE_SCANNING) {
            // 当前正在扫描中，做UI层的相应的处理
          } else if (state == BleEventConstants.SCAN_STATE_IDLE) {
            // 当前扫描结束，做UI层的相应的处理
          }
        });
      },
      onError: (error) {
        log("Scan state stream error: $error");
      },
    );
  }
```

2.3 扫描设备列表流

```
  static Stream<List<ScanDevice>> get scanDeviceListStream {
    return baseStream
        .where((event) => event is Map && event[BleEventConstants.KEY_TYPE] ==
BleEventConstants.TYPE_SCAN_DEVICE_LIST)
        .map((event) {
      final list = event[BleEventConstants.KEY_VALUE][BleEventConstants.KEY_LIST] as List?
?? [];
      return list
          .whereType<Map>()
          .map((deviceMap) => ScanDevice.fromMap(deviceMap))
          .toList();
    });
  }
```

```
使用示例：
  List<ScanDevice> _devices = [];
  StreamSubscription<List<ScanDevice>>? _scanSubscription;

  List<ScanDevice> convertToScanDeviceList(List<dynamic> list) {
  return list.map((item) {
    if (item is ScanDevice) {
      return item;
    } else if (item is Map) {
      return ScanDevice.fromMap(item);
    } else {
      throw Exception('无法转换的类型：${item.runtimeType}');
    }
  }).toList();
}

void _subscribeToScanListStream() {
  _scanSubscription = BleEventStream.scanDeviceListStream.listen((devices) {
    setState(() => _devices = convertToScanDeviceList(devices));
  }) as StreamSubscription<List<ScanDevice>>?;
}
```

2.4 设备连接状态流

```
static Stream<DeviceConnection> get deviceConnectionStream {
  return baseStream
      .where((event) {
    return event is Map && event[BleEventConstants.KEY_TYPE] ==
BleEventConstants.TYPE_DEVICE_CONNECTION;
  })
      .map((event) {
    final data = event[BleEventConstants.KEY_VALUE];
    return DeviceConnection.fromMap(data);
  });
}

使用示例：
StreamSubscription<DeviceConnection>? _deviceConnectionSubscription;

void _subscribeToDeviceConnectionStream() {
  _deviceConnectionSubscription = BleEventStream.deviceConnectionStream
      .listen(
        (connection) async {
      log("Device connection status: ${connection.state}");

      if (connection.state == AppConstants.connectionConnecting) {
        // 进行设备连接中的UI状态处理
      } else {
        // 进行设备非连接中的UI状态处理
      }
```

```
      },
      onError: (error) {
        log("Device connection stream error: $error");
      },
    ) as StreamSubscription<DeviceConnection>?;
  }
```

2.5 OTA连接状态流

```
  static Stream<Map<String, dynamic>> get otaConnectionStream {
    return baseStream
        .where((event) => event is Map && event[BleEventConstants.KEY_TYPE] ==
BleEventConstants.TYPE_OTA_CONNECTION)
        .map((event) {
      final data = event[BleEventConstants.KEY_VALUE];
      return {
        BleEventConstants.KEY_STATE: data[BleEventConstants.KEY_STATE],
        BleEventConstants.KEY_DEVICE_TYPE: data[BleEventConstants.KEY_DEVICE_TYPE],
      };
    });
  }

  使用示例:
  StreamSubscription<Map<String, dynamic>>? _otaConnectionSubscription;

  void _subscribeToOtaConnectionStream() {
    _otaConnectionSubscription = BleEventStream.otaConnectionStream.listen(
      (otaData) {
        if (mounted) {
          // 处理otaData, 并进行UI状态的更新
        }
      },
      onError: (error) {
        log("OTA connection stream error: $error");
      },
    );
  }
```

2.6 日志文件列表流

```
  static Stream<List<Map<String, String>>> get logFilesStream {
    return baseStream
        .where((event) =>
    event is Map &&
        event[BleEventConstants.KEY_TYPE] == BleEventConstants.TYPE_LOG_FILES)
        .map((event) {
      final files = event[BleEventConstants.KEY_FILES] as List? ?? [];
      return files
```

```
          .whereType<Map>()
          .map((fileMap) =>
      {
        BleEventConstants.KEY_NAME: fileMap[BleEventConstants
            .KEY_NAME] as String? ?? '',
      }).toList();
    });
  }

  使用示例:
  List<Map<String, String>> _logFileList = [];
  StreamSubscription? _logFileListSubscription;

  void _subscribeToLogFileListStream() {
    _logFileListSubscription = BleEventStream.logFilesStream.listen(
          (logFileList) {
        setState(() {
          _logFileList = logFileList;
        });
      },
      onError: (error) {
        log("Error listening to logFilesStream: $error", error: error);
      },
    );
  }
```

2.7 日志文件详情流

```
  static Stream<String> get logDetailFilesStream {
    return baseStream
        .where((event) => event is Map && event[BleEventConstants.KEY_TYPE] ==
BleEventConstants.TYPE_LOG_DETAIL_FILES)
        .map((event) {
      final files = event[BleEventConstants.KEY_FILES] as List? ?? [];
      return files.first as String? ?? '';
    });
  }

  使用示例:
  StreamSubscription? logDetailSubscription;
  String logDetailTxt = '';

    logDetailSubscription = BleEventStream.logDetailFilesStream.listen(
      (logDetail) {
        setState(() {
          logDetailTxt = logDetail;
        });
      },
      onError: (error) {
        log("Error listening to logDetailFilesStream: $error");
```

```
        logDetailSubscription?.cancel();
      },
    );
```

2.8 下载状态流

```
  static Stream<Map<String, dynamic>> get downloadStatusStream {
    return baseStream
        .where((event) => event is Map && event[BleEventConstants.KEY_TYPE] ==
BleEventConstants.TYPE_DOWNLOAD_STATUS)
        .map((event) {
      final data = event[BleEventConstants.KEY_VALUE];
      return {
        BleEventConstants.KEY_STATUS: data[BleEventConstants.KEY_STATUS],
        BleEventConstants.KEY_PROGRESS: data[BleEventConstants.KEY_PROGRESS],
        BleEventConstants.KEY_MESSAGE: data[BleEventConstants.KEY_MESSAGE],
      };
    });
  }

  使用示例:
  StreamSubscription? _downloadStatusSubscription;

  void _startListeningToDownloadStatus() {
    _downloadStatusSubscription = BleEventStream.downloadStatusStream.listen((data) {
      setState(() {
        var state = data[BleEventConstants.KEY_STATUS];
        if (state == BleEventConstants.STATUS_ON_STOP || state ==
BleEventConstants.STATUS_ON_ERROR) {
          if (mounted) {
            if (Navigator.of(context).canPop()) {
              Navigator.of(context).pop();
              AppUtil.readFileList();
            }
          }
        }
        final progressValue = data[BleEventConstants.KEY_PROGRESS];
        if (state == BleEventConstants.STATUS_ON_PROGRESS && progressValue != null) {
          // 得到progressValue显示在UI上
        }
      });
    });
  }
```

2.9 OTA文件列表流

```
  static Stream<List<Map<String, String>>> get otaFileListStream {
    return baseStream
```

```
        .where((event) => event is Map && event[BleEventConstants.KEY_TYPE] ==
BleEventConstants.TYPE_OTA_FILE_LIST)
        .map((event) {
      final list = event[BleEventConstants.KEY_VALUE][BleEventConstants.KEY_LIST] as List?
?? [];
      return list
          .whereType<Map>()
          .map((fileMap) => {
        BleEventConstants.KEY_NAME: fileMap[BleEventConstants.KEY_NAME] as String? ?? '',
        BleEventConstants.KEY_PATH: fileMap[BleEventConstants.KEY_PATH] as String? ?? '',
      })
          .toList();
    });
  }

  使用示例:
  StreamSubscription<List<Map<String, String>>>? _otaFileListSubscription;
  List<Map<String, String>> _otaFileList = []; // 用于存储文件列表

  void _startListeningToOtaFileList() {
    _otaFileListSubscription?.cancel();
    _otaFileListSubscription = BleEventStream.otaFileListStream.listen((
      fileList,
    ) {
      if (mounted) {
        setState(() {
          _otaFileList = fileList;
        });
      }
    });
  }
```

2.10 强制升级流

```
  static Stream<bool> get mandatoryUpgradeStream {
    return baseStream
        .where((event) => event is Map && event[BleEventConstants.KEY_TYPE] ==
BleEventConstants.TYPE_MANDATORY_UPGRADE)
        .map((event) {
      final data = event[BleEventConstants.KEY_VALUE];
      return data[BleEventConstants.KEY_IS_REQUIRED] as bool;
    });
  }

  使用示例:
  StreamSubscription<bool>? _mandatoryUpgradeSubscription;
  _mandatoryUpgradeSubscription = BleEventStream.mandatoryUpgradeStream
        .listen((isRequired) {
          if (isRequired && mounted) {
            setState(() {
```

```
            ToastUtils.show(
              context,
              AppLocalizations.of(context)!.deviceMustMandatoryUpgrade,
            );
          });
        }
      });
```

2.11 OTA状态流

```
static Stream<Map<String, dynamic>> get otaStateStream {
  return baseStream
      .where((event) =>
  event is Map && event[BleEventConstants.KEY_TYPE] == BleEventConstants.TYPE_OTA_STATE)
      .map((event) {
    final data = event[BleEventConstants.KEY_VALUE];
    final state = data[BleEventConstants.KEY_STATE];
    final result = {
      BleEventConstants.KEY_STATE: state,
      BleEventConstants.KEY_SUCCESS: data[BleEventConstants.KEY_SUCCESS],
      BleEventConstants.KEY_CODE: data[BleEventConstants.KEY_CODE],
      BleEventConstants.KEY_TYPE: data[BleEventConstants.KEY_TYPE],
      BleEventConstants.KEY_MESSAGE: data[BleEventConstants.KEY_MESSAGE],
    };
    if (state == BleEventConstants.KEY_STATE_WORKING) {
      result[BleEventConstants.KEY_PROGRESS] = data[BleEventConstants.KEY_PROGRESS];
    }
    return result;
  });
}

使用示例:
StreamSubscription? _otaStateSubscription;

void _startListeningToOtaState() {
  _otaStateSubscription = BleEventStream.otaStateStream.listen((otaData) {
    if (mounted) {
      setState(() {
        updateOtaState(otaData);
      });
    }
  });
}

void updateOtaState(Map<String, dynamic> otaData) {
  if (!mounted) return;

  final newOtaState =
      otaData[BleEventConstants.KEY_STATE] as String? ??
          BleEventConstants.KEY_STATE_UNKNOWN;
```

```
    setState(() {
      _otaState = newOtaState;

      // 处理不同状态的数据
      switch (_otaState) {
        case BleEventConstants.KEY_STATE_WORKING:
          // 处理OTA升级中的UI逻辑
          break;
        case BleEventConstants.KEY_STATE_IDLE:
          // 处理OTA升级完成的UI逻辑
          break;
      }
    });
  }
```